INSTITUT NATIONAL DES SCIENCES APPLIQUÉES LYON

Group: Hagemann Julian, Nguyen Sang, Vivier Thibault, Tron Baptiste

Cryptography & Blockchain - DApp Project Report

1. Introduction **2**
1.1 Description of the Idea and Purpose of the Application 2
1.2 Justification for the Use of Blockchain Technology 3
1.3 Limitations of Non-Blockchain Approaches 4
2. Technologies **5**
2.1 Description of the Use of ERC Standards 5
2.2 Description of the Use of Cryptography in the Application 5
2.3 Description of the Utilization of Additional Blockchain Technologies 6
2.3.1 Oracle Mechanism for Event Resolution 6
2.3.2 Future Outlook: External Oracles and Automated Market Makers (AMMs) 7
3. Specifications **9**
3.1 Use Case Diagram 9
3.2 Description of Each Major Use Case 9
3.2.1 Use Case 1 - Buy Shares / Placing a Bet 9
3.2.2 Use Case 2 - View Market Details and Implied Odds 10
3.2.3 Use Case 3 - Create Market 10
4. Design **11**
4.1 Smart Contract 11
4.1.1 Object and Structures 11
4.1.2 Functions 11
4.2 Diagram Showing Sequence of Events for Use Case 1 (Buy Shares / Placing Bet) 13

1. Introduction

1.1 Description of the Idea and Purpose of the Application

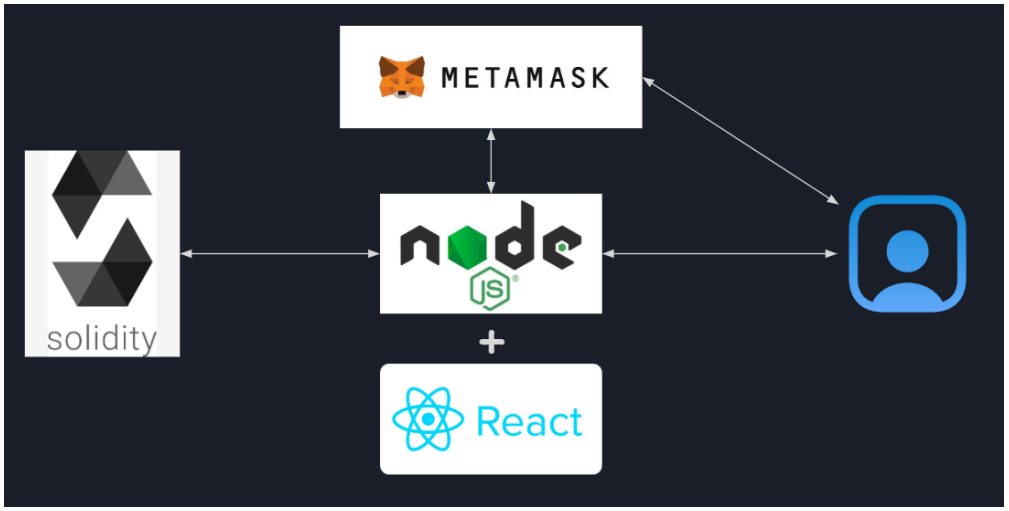
Forecasting real-world events often requires gathering information from many independent participants. Prediction markets offer a mechanism to achieve this, but traditional platforms typically depend on centralized middlemen that control funds and outcomes.

The goal of this project is to design and implement a prediction market as a decentralized application (DApp). The application allows users to create prediction markets on real-world or predefined events and to place bets on the possible outcomes using a dedicated cryptographic token. Once the event is resolved, users who predicted the correct outcome are rewarded proportionally to their stake.

Unlike traditional betting platforms, this prediction market is fully decentralized and operates through smart contracts deployed on the Ethereum blockchain. All critical logic, including market creation, token transfers, bet placement, event resolution, and payout distribution (the amount of money being paid after event resolution), is enforced by immutable smart contracts. This ensures that no single authority can manipulate outcomes, censor users, or alter payouts after bets have been placed.

The application is composed of two main components:

- Smart contracts that implement the prediction market logic and manage funds and outcomes.
- A frontend web interface that allows users to interact with the blockchain in a user-friendly way.



The primary purpose of the application is to demonstrate how blockchain technology can be used to create a trustless, transparent, and verifiable market mechanism in which users can make binding predictions without relying on a central operator.

Event resolution is handled through an on-chain oracle mechanism detailed in the Technologies section.

1.2 Justification for the Use of Blockchain Technology

Blockchain is essential for prediction markets because it removes the need to trust a central authority. In centralized systems, users must rely on the operator to manage bets, resolve outcomes, and distribute rewards fairly, creating risks of manipulation or failure.

By using blockchain technology and smart contracts, these trust assumptions are removed. The prediction market smart contract acts as an impartial and autonomous entity that enforces the rules of the market exactly as programmed. Once deployed, the contract cannot be altered, ensuring that the logic governing bets and payouts is immutable and verifiable by any participant.

Additionally, blockchain provides:

- Transparency, as all transactions, bets, and outcomes are publicly auditable.
- Integrity, since cryptographic mechanisms prevent unauthorized modification of data.
- Guaranteed outcomes, as smart contracts automatically execute payouts once the resolution conditions are met.
- Binding commitments, because funds are locked in the contract and cannot be withdrawn arbitrarily.

The use of an ERC-compliant token enables standardized, secure value transfer between users and smart contracts. Cryptographic primitives such as hashing and digital signatures are leveraged to authenticate users and ensure the integrity of interactions with the system.

1.3 Limitations of Non-Blockchain Approaches

Developing this application using non-blockchain technologies would be difficult and fundamentally different in terms of trust and guarantees. In a centralized architecture, the backend server would need to:

- Hold and manage all user funds
- Resolve events and decide winners
- Enforce payouts and balances

This would require users to fully trust the operator of the system, as the backend could arbitrarily change results, delay payouts, or misuse funds. Even with strong security measures, centralized systems remain vulnerable to insider attacks, server compromises, and regulatory or political censorship.

Moreover, ensuring transparency and verifiability in a centralized system is challenging, as users cannot independently verify that the rules were enforced correctly. Any audit would depend on logs and data controlled by the same authority operating the system.

In contrast, blockchain-based smart contracts provide a shared source of truth that is independent of any single entity. The decentralized execution model ensures that all users interact under the same rules, and cryptographic verification replaces institutional trust. For these reasons, blockchain technology is not only suitable but necessary to achieve the goals of a decentralized and trustless prediction market.

2. Technologies

2.1 Description of the Use of ERC Standards

The application makes use of the **ERC-20 token** standard to represent and transfer value within the prediction market. An ERC-20 compliant token is deployed and used as the sole medium for placing bets, locking funds, and distributing rewards. By relying on a standardized token, the application ensures compatibility with common Ethereum tooling and wallets, such as MetaMask, and simplifies interaction between users and smart contracts.

Participants must hold the ERC-20 token in order to interact with the prediction market. When a user places a bet on a specific outcome, the corresponding amount of tokens is transferred from the user's address to the prediction market smart contract and locked until the market is resolved. Once the final outcome is determined by the oracle mechanism, the smart contract automatically redistributes the pooled tokens to users who predicted the correct outcome, proportionally to their stake.

Using the ERC-20 standard provides several advantages. First, it guarantees a well-defined and secure interface for token transfers and balance management, reducing the risk of implementation errors. Second, it enables transparent and auditable tracking of all value flows within the system, as token transfers are recorded on the blockchain. Finally, it enforces binding commitments, as users cannot withdraw or reuse tokens once they have been committed to a market, ensuring fairness and integrity in the betting process.

2.2 Description of the Use of Cryptography in the Application

Cryptographic mechanisms play a fundamental role in ensuring the security, integrity, and trustlessness of the prediction market application. The system relies on the cryptographic primitives provided by the Ethereum blockchain to authenticate users, protect data integrity, and enforce correct execution of the market logic.

User authentication and authorization are based on public-key cryptography. Each participant interacts with the application through an Ethereum account controlled by a private key. Transactions such as placing bets, proposing outcomes, or claiming rewards must be

digitally signed by the user, ensuring that only the rightful owner of an account can initiate actions on behalf of that address.

Cryptographic hashing is used extensively within the blockchain to ensure data integrity and immutability. All transactions, smart contract states, and event logs are hashed and linked within the blockchain, making any tampering immediately detectable. This guarantees that market parameters, bets, and outcomes remain transparent and verifiable for all participants.

Additionally, cryptography underpins the enforcement of trustless execution through smart contracts. Once deployed, the smart contract bytecode is immutable, and its execution is deterministically verified by all network nodes. This ensures that the rules governing market creation, betting, oracle resolution, and payout distribution are executed exactly as defined, without the possibility of interference by a centralized authority.

Finally, cryptography is used to secure the bet resolutions. Everyone can call the resolution function of the smart contract, but only the Oracle is able to resolve the bets by signing messages with its private key. The smart contract will then approve the bet resolution based on authenticity verification. This protects the message, and also the bet resolution function. With this mechanism, only the Oracle is able to resolve the bets.

Together, these cryptographic foundations enable the application to replace institutional trust with mathematical guarantees, ensuring that all participants interact under the same verifiable rules and that the outcomes of prediction markets are both secure and transparent.

2.3 Description of the Utilization of Additional Blockchain Technologies

2.3.1 Oracle Mechanism for Event Resolution

Prediction markets require a reliable mechanism to resolve real-world events in a trustless way. Since smart contracts cannot directly access external information, this application relies on an on-chain oracle mechanism to determine the outcome of each prediction market.

In this project, we choose to implement a centralized Oracle locally for development simplicity and keep the app free to develop. Using an external decentralized Oracle is subject to fees. In our implementation, the admin is considered as the Oracle, and users have to trust him. But, the application is designed to allow a replacement of the Oracle

system by an external decentralized one, which will be preferred if the application is then deployed afterward.

As an alternative to a centralized oracle, we also considered implementing a local decentralized optimistic oracle based on a simple dispute-driven mechanism. After a market reaches its closing time, any participant may propose the outcome of the event by submitting it to the smart contract accompanied by a stake. The proposed outcome is accepted optimistically and assumed to be correct by default, but is subjected to a predefined challenge period. During this challenge window, other participants may dispute the proposed outcome by submitting a counter-stake if they believe the proposal to be incorrect. If no challenge is raised within the allotted time, the proposed outcome is automatically finalized by the smart contract and used to trigger the payout logic. In the event of a challenge, the contract transitions into a dispute phase in which a predefined resolution mechanism is invoked, such as a committee-based or vote-based decision process. A diagram can be found in section 4.2. This approach effectively implements a self-contained oracle mechanism that avoids granting unilateral control over event resolution to a single party, while keeping the common-case resolution process efficient, low-cost, and suitable for a local deployment environment. However, the correctness of the final outcome ultimately depends on the assumption that participants are incentivized to challenge incorrect proposals and that disputes are resolved honestly.

2.3.2 Future Outlook: External Oracles and Automated Market Makers (AMMs)

In the future, the oracle mechanism could be extended or replaced by integration with established external oracle networks such as Universal Market Access (UMA) or Chainlink. These systems provide decentralized and economically secured data feeds that are widely used in real-world and decentralized finance applications. Integrating such would reduce reliance on central or local dispute resolution mechanisms, improve trust when resolving real-world events. Deploying the application on a public Ethereum-compatible network would be necessary for this interaction with external oracle infrastructure.

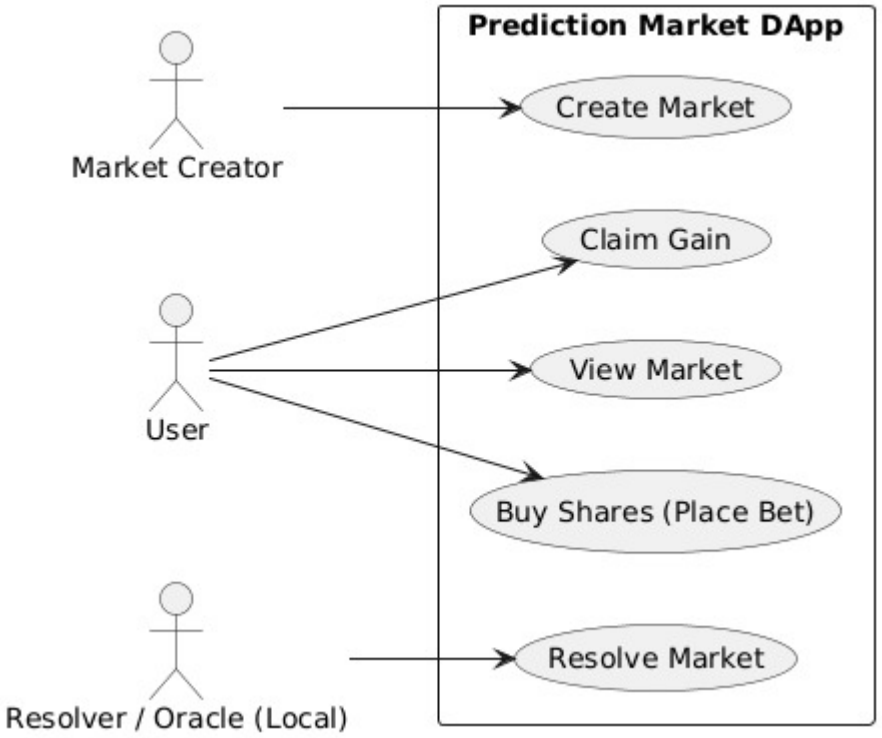
While the current implementation relies on a share-based betting model, future extensions could adopt alternative representations for user positions (Option A or Option B). For example, while retaining fungible tokens as the currency used to place bets, outcome positions themselves could be represented using a tokenized position

model (e.g. using the ERC-1155 multi-token standard). In this design, each outcome position would be represented by a transferable token that encodes the right to a payout if the corresponding outcome is deemed correct. This would allow outcome positions to be independently held and transferred, enabling more flexible interaction with prediction markets, including the ability to trade positions prior to market resolution.

Combined with the integration of an Automated Market Maker (AMM) or an order-book-based trading mechanism to trade these positions, such a design would allow users to dynamically enter and exit positions before market resolution. This would support dynamic price discovery, improve liquidity and enable more advanced trading strategies such as partial exits or hedging, while providing more accurate implied probabilities for event outcomes.

3. Specifications

3.1 Use Case Diagram



3.2 Description of Each Major Use Case

3.2.1 Use Case 1 - Buy Shares / Placing a Bet

- Primary actor: User
- Preconditions: The market is open; the user has sufficient token balance and allowance.
- Main flow:
    1. The user selects a market, chooses an outcome, and specifies the amount to bet.
    2. The frontend submits a transaction to the smart contract.
    3. The smart contract transfers tokens from the user and records the shares for the chosen outcome.
    4. Outcome totals are updated accordingly.

- Postconditions: The user's tokens are locked in the contract, and their shares contribute to the selected outcome.

3.2.2 Use Case 2 - View Market Details and Implied Odds

- Primary actor: User
- Preconditions: A market exists on the blockchain.
- Main flow:
    1. The user selects a specific market.
    2. The frontend queries the smart contract to obtain outcome options and current share totals.
    3. The application computes implied odds based on the distribution of shares for each outcome.
- Postconditions: No state changes occur; the user can assess market information and implied probabilities before betting.

3.2.3 Use Case 3 - Create Market

- Primary actor: Market Creator
- Preconditions: The creator is authorized to create markets; market parameters are valid.
- Main flow:
    1. The creator specifies the market question, possible outcomes, and closing time.
    2. The frontend submits a transaction to the smart contract to create the market.
    3. The smart contract stores the market data and emits a creation event.
- Postconditions: A new market is created in an open state and becomes visible to all users.

4. Design

4.1 Smart Contract

The project is based on a single Smart contract: PredictionMarket.sol.

4.1.1 Object and Structures

- Stage : “Active”, “Pending”, “Resolved” : It corresponds to the current state of a Market. It determines where the Market will be shown in the application and which operations can be performed on it.

- Market is the main object and is composed of:
    - id
    - title
    - Option A : first possible choice
    - Option B : second possible choice
    - end_time: A countdown in seconds which determines when the Market ends.
    - pool_A: the amount placed on the Option A.
    - pool_B: the amount placed on the Option B.
    - stage: the first object detailed sooner.
    - wining_outcome: Which side has won?
    - creator: the address of the creator.

- Bet is related to a market and is composed of:
    - amount: in ETH.
    - choice: Option A or B.
    - exists: to manage the right to bet only in on choice, or to overbet (add a bigger amount on the option chosen).

4.1.2 Functions

There are four main functions in the smart contract: createMarket, placeBet, resolveMarket, and claimGain. Each of those functions is essential to the application. They are following a logical guideline.

- createMarket: To be able to bet, users need to have access to Markets. These functions allow every user to create their own Markets, which can then be seen by

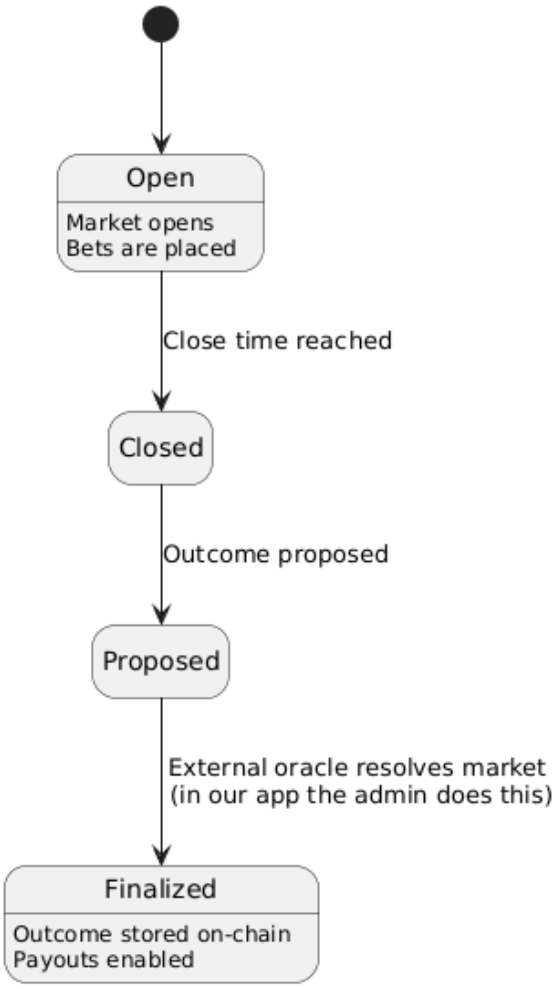
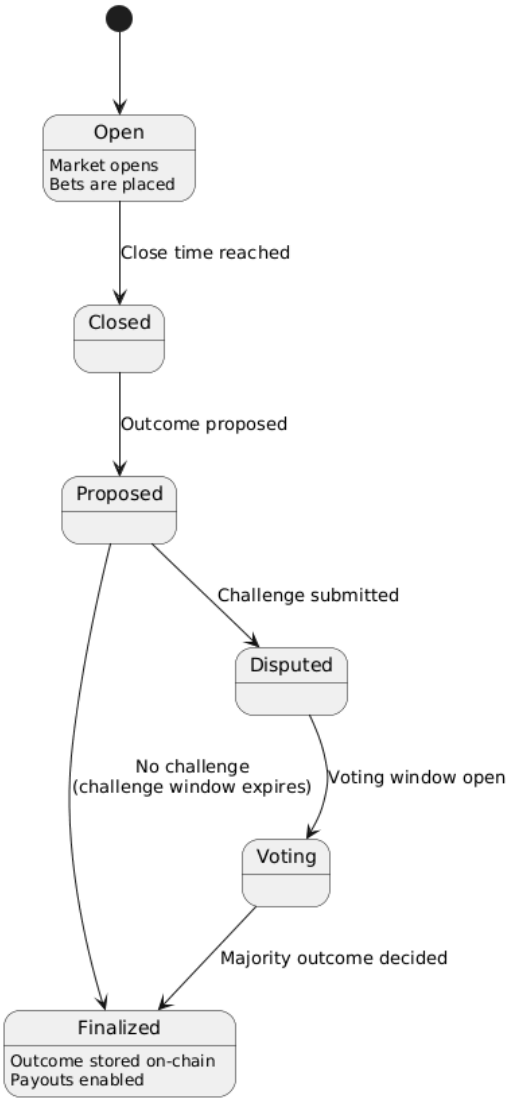
everyone in the Global window of the application. To create a Market, the user must specify the main characteristics of the Market object (title, option A, option B, end_time, …)

- placeBet: Then, when a Market is created, it is possible to place a bet on it. Users must choose an Option to bet on, and decide on the bet amount (in ETH). Cross betting is not allowed, which means it is not possible to bet on option A and option B for the same Market.

- resolveMarket: When the end_time comes to its end, the market status becomes “Pending”. It had to be resolved. This function allows everyone to make a proposal of resolution. But only the Oracle one can resolve the bets, because of the authentication process implemented using cryptography. This mechanism protect the bets resolution (cf. 2.2). The Oracle mechanism is not in the smart contract by choice, this design allows us to replace the current local and centralized Oracle by an external decentralized one.

- claimGain: Depending on the bet outcome, the user may win or lose. The winners must be able to claim their gains and get the ETH reward in their wallet.

.

4.2 Diagram Showing Sequence of Events for Use Case 1 (Buy Shares / Placing Bet)

with external or admin oracle integration	with local decentralized optimistic oracle (as described in section 2.3.1)